TECHNICAL PROJECT REPORT

ARDUINO-BASED GESTURE-CONTROLLED ROBOTIC ARM

A microcontroller-based robotic system that uses hand-gesture sensing and wireless communication to control servo-driven arm movements.

SYSTEM OVERVIEW

Core elements documented in the report:
Arduino Nano | MPU6050 motion sensor | HC-05 Bluetooth modules | SG90 servo motors

ANONYMIZED PUBLIC EDITION

Academic team project - June 2023
Contributor names and institutional identifiers have been removed for this public copy. This report does not claim sole authorship.

Abstract

This project aims to design and develop an Arduino-based gesture controlled robotic arm. The project utilizes Arduino nano ,MPU 6050 ,Bluetooth modules(hc-05) and servo motors to create a versatile and programmable arm capable of performing precise and controlled movements. The mechanical structure of the arm is carefully designed and constructed to ensure stability and flexibility. The integration of sensors enables the arm to perform tasks which are instructed through the controller.The project emphasizes affordability and adaptability, highlighting the potential of Arduino as a cost-effective platform for constructing robotic systems. The robotic arm serves as a practical demonstration of Arduino's capabilities and opens up possibilities for educational purposes and real-world applications in industries requiring automation and manipulation of objects.

Contents

Abstract ..... **i**
Contents ..... **ii-iii**
Chapter 01: Introduction **1**
1.1. Introduction 1
1.2. Background 1
1.3. Project Objectives 2
1.4. Scope 2
1.5. Project Management 3
1.6. Overview and Benefits 4
1.7. Organization of the Report 5
Chapter 02: Background Review & Survey **6**
2.1. Related Works 6-7
Chapter 03: Theoretical Aspects **8**
3.1. Introduction to Robotic Arm 8-9
3.2. Features of Robotic Arm 9
3.3. Advantages of Robotic Arm 9-10
3.4. Disadvantages of Robotic Arm 10-11
3.5. Application areas of Robotic Arm 11
3.6. Robotic Arm Technologies 11-12
3.7. Project Layout 12
Chapter 04: Hardware Requirements **13**
4.1. Arduino Nano 13
4.1.1. Features 13
4.1.2. Pin Configuration 13-14
4.2. Servo Motor (SG90) 14
4.2.1. Features 14
4.2.2. Pin Configuration 14
4.3. Block diagram of the proposed system 15
4.5.1. Working of the system 15-16
4.5.2. Components Required 16

Chapter 05: Software Requirements **17**
5.1. Arduino IDE (Embedded C / C++) 17
5.2. Logic and Flowchart 18
Chapter 06: Project development & Testing Aspects **19**
6.1. 19
Chapter 07: Conclusion & Future Scope **20**
7.1. Result 20
7.2. Conclusion 20
7.3. Limitations 20
7.4. Further Enhancement and Future Scope 21
References **22**
Appendix 01 ..... **23**
A01.1. Code Listing 23
A01.2. Main Code 23-36
A01.3. Libraries ..... .36
Appendix 02 ..... **37**
A02.1. Project Proposal Form 37
A02.2. Project Management 37
A02.3. Bill of Material 37-38
Appendix 03 ..... **39**
A03.1. Data Sheets 39-42

Chapter 01: Introduction

1.1. Introduction

The Arduino-based robotic arm is an innovative and versatile device designed to mimic human arm movements and perform various tasks with precision. Powered by Arduino technology, this robotic arm offers a flexible and programmable platform for automation enthusiasts and hobbyists. With multiple degrees of freedom and a sturdy construction, it can manipulate objects, perform complex motions, and execute pre-programmed sequences. Equipped with sensors and actuators, it can interact with its environment, making it suitable for applications in education, research, and industrial automation. The Arduino platform provides an intuitive and user-friendly interface for programming and controlling the robotic arm, allowing users to explore endless possibilities in robotics and automation. Whether for educational purposes or practical applications, the Arduino-based robotic arm opens up a world of possibilities for robotic enthusiasts.

However here we are using a sensor enabled glove with which we are controlling the movements of the joints. The glove will measure the movements and will send the same to the arm via Bluetooth .

1.2. Background

The development of Arduino-based robotic arms emerged from the growing interest in robotics and automation. Arduino, an open-source electronics platform, provided an accessible and affordable option for enthusiasts and developers to create programmable robotic systems. Robotic arms, known for their precision and versatility, became a popular focus within this community. By integrating Arduino technology, developers were able to design robotic arms with multiple degrees of freedom, allowing for intricate movements and object manipulation. The Arduino platform facilitated easy programming and control of these arms, enabling users to create custom sequences and automate various tasks. This combination of Arduino and robotic arm technology has sparked a revolution in DIY robotics, empowering individuals to explore the realms of automation, engineering, and innovation. Today,

Arduino-based robotic arms continue to evolve, finding applications in fields such as education, research, manufacturing, and even home automation.

1.3. Project Objectives

The project objectives of an Arduino-based robotic arm can include:

1. Automation: The primary objective of an Arduino-based robotic arm project is to achieve automation in various tasks by providing a programmable and adaptable robotic arm capable of performing precise movements and actions.

2. Manipulation: The robotic arm aims to manipulate objects with dexterity and precision, allowing for tasks such as picking and placing objects, assembly, sorting, or even delicate operations like surgery or laboratory experiments.

3. Education and Learning: The project seeks to provide an educational platform for individuals to learn and understand robotics, programming, and automation principles through hands-on experience with the Arduino-based robotic arm.

4. Customizability: The robotic arm aims to be highly customizable, allowing users to modify and adapt its design, control algorithms, and functionality to suit their specific requirements or experiment with different sensors and actuators.

5. Accessibility: The project aims to make robotics and automation accessible to a wider audience by using the Arduino platform, which is known for its user-friendly interface, affordability, and extensive community support, thus encouraging innovation and collaboration.

1.4. Scope

The scope of an Arduino-based robotic arm extends to various fields such as education, research, industrial automation, and hobbyist projects. It can be used for tasks like object manipulation, assembly, pick-and-place operations, and more. The arm's programmability and flexibility make it suitable for learning robotics and automation principles. It can also be integrated into industrial settings to automate repetitive tasks, improve efficiency, and reduce human error. Additionally, the open-source nature of Arduino allows for collaboration and innovation, expanding the scope for customized applications and advancements in robotic arm technology.

1.5. Project Management

According to the PMBOK Guide (Project Management Body of Knowledge), a project management life cycle consists of 5 distinct phases including initiation, planning, execution, review, and closure that combine to turn a project idea into a working product.

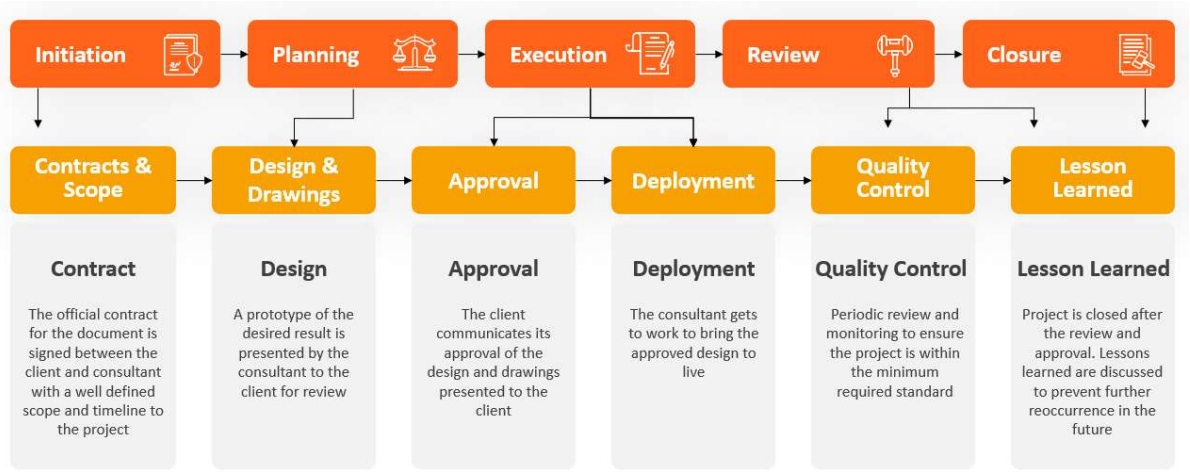


Figure 1. Model of phases in project management.

The project initiation phase is the first stage of turning an abstract idea into a meaningful goal. In this stage, we need to develop a business case and define the project on a broad level.

The project planning stage requires complete diligence as it lays out the project's roadmap.

The project execution stage is where the project team does the actual work. The job of a project manager is to establish efficient workflows and carefully monitor the progress of the team.

In the project management process, the third and fourth phases are not sequential in nature. The project monitoring and controlling phase run simultaneously with project execution.

The project closure stage indicates the end of the project after the final delivery.

1.6. Overview and Benefits

The Arduino-based robotic arm offers an advanced and customizable solution for automation tasks, education, research, and hobbyist projects. It combines the power of Arduino technology with the precision and versatility of a robotic arm, opening up a wide range of possibilities.

The robotic arm's programmability allows users to define and execute complex sequences of movements, making it ideal for tasks like object manipulation, pick-and-place operations, assembly, and more. It can be integrated into various industries to automate repetitive processes, improve efficiency, and reduce human error, ultimately enhancing productivity.

In educational settings, the Arduino-based robotic arm serves as a valuable tool for teaching robotics, programming, and automation principles. Students can gain hands-on experience, develop problem-solving skills, and explore the intersection of engineering and robotics.

The open-source nature of Arduino fosters a collaborative community where users can share ideas, designs, and code, facilitating innovation and advancements in robotic arm technology. Additionally, the affordability and accessibility of Arduino make the robotic arm more attainable for enthusiasts, researchers, and small-scale applications.

Overall, the Arduino-based robotic arm provides an intuitive platform that combines flexibility, programmability, and affordability, making it an excellent choice for a wide range of applications, from educational to industrial automation.

1.7. Organization of the Report

The report has been made and organized into various chapters with different theory and information in each chapter.

Chapter 1 gives basic introduction , background of the project, scope of the project in future.

Chapter 2 gives the information about the background reviews and surveys done to analyse the usage and demand of the project.

Chapter 3 gives all the necessary theoretical knowledge about the project.

Chapter 4 deals with the hardware requirements and circuit diagram used to build the project.

Chapter 5 deals with the software requirements to calibrate the project to do the required output working.

Chapter 6 gives the information about the stages of project development and testing aspects.

Chapter 7 deals with the conclusion of the project and discussion of its future scope in a much elaborated manner.

Chapter 02: Background Review & Survey

2.1. Related Works

Robotic arms have been a popular area of research for many years due to their potential for use in a variety of applications, including manufacturing, agriculture, medical procedures, and space exploration. Here are some of the most notable and recent works related to robotic arms:

"A Comprehensive Review of Robotic Arm Control Strategies" (2021)

This review paper discusses various control strategies used in robotic arms, including kinematic and dynamic control, model-based and model-free control, and hybrid control. The paper also discusses the advantages and limitations of each strategy and their suitability for different applications.

"Design and Control of a Robotic Arm for Industrial Applications" (2020)

This research paper presents the design and control of a 5-DOF robotic arm for industrial applications. The authors use a combination of kinematic and dynamic control techniques to achieve accurate and smooth motion control. The arm is tested for pick and place operations in an industrial setting.

"Robotic Arm for Precision Agriculture" (2021)

This paper presents the design and implementation of a robotic arm for precision agriculture. The arm is designed to perform tasks such as plant pruning, fruit picking, and weed removal. The authors use a combination of computer vision and force sensing to control the arm's motion and ensure accurate and gentle handling of crops.

"Development of a Robotic Arm for Medical Applications" (2020)

This research paper presents the development of a robotic arm for medical applications such as laparoscopic surgery. The arm is designed to be compact and

lightweight, with high precision and dexterity. The authors use a combination of kinematic and force control to achieve accurate and safe motion control.

"Design and Control of a Robotic Arm for Space Applications" (2021)

This paper presents the design and control of a robotic arm for space applications, such as servicing of satellites and other space objects. The arm is designed to be lightweight, compact, and highly maneuverable. The authors use a combination of kinematic and dynamic control techniques to achieve accurate and robust motion control in micro-gravity environments.

Overall, these works demonstrate the versatility and potential of robotic arms in various applications. The development of new control strategies, sensing technologies, and materials will likely lead to even more sophisticated and capable robotic arms in the future.

Chapter 03: Theoretical Aspects

3.1. Robotic Arm

The theoretical aspects of an Arduino-based robotic arm encompass a range of principles and concepts that form the foundation for its design, control, and operation.

1. Kinematics: Kinematics is a fundamental aspect of the robotic arm, focusing on the study of its motion, position, and orientation in three-dimensional space. Forward kinematics determines the end effector's position and orientation based on the joint angles, while inverse kinematics involves calculating the joint angles required to achieve a desired end effector position or orientation. Mathematical models, such as Denavit-Hartenberg parameters or transformation matrices, are used to describe the arm's kinematic relationships.

2. Control Systems: Control systems are vital for precise motion control of the robotic arm. These systems involve implementing control algorithms to regulate the joint angles and achieve accurate movement. Proportional-Integral-Derivative (PID) control is commonly used, adjusting the motor or actuator inputs based on feedback from sensors to minimize errors between the desired and actual positions. Advanced control techniques, such as adaptive control or model predictive control, can be explored for more complex applications.

3. Actuation: Actuation refers to the selection and control of motors or actuators responsible for moving the robotic arm's joints. The choice of actuators depends on factors such as torque requirements, speed, and precision. Servo motors are commonly used for smaller robotic arms, providing precise control and position feedback, while larger arms may use stepper motors or even hydraulic/pneumatic actuators for higher power and payload capabilities.

4. Programming: The robotic arm is programmed using a suitable programming language, such as C/C++, to define tasks, control sequences, and interaction with other devices or systems. Programming includes coding the algorithms for motion control, sensor integration, collision avoidance, and any specialized functionalities required for the specific application. The Arduino platform offers

a user-friendly programming environment, libraries, and APIs that simplify development.

5. Sensor Integration: Sensors play a critical role in the operation of the robotic arm, providing feedback on joint positions, velocities, forces, and environmental conditions. Encoders, potentiometers, or resolvers are commonly used to measure joint angles. Here we have used MPU 6050 to sense acceleration and movement of the hand and send the readings to the arm .

Understanding these theoretical aspects is crucial for designing, controlling, and programming an Arduino-based robotic arm effectively. It allows developers and enthusiasts to optimize the arm's performance, adapt it to different tasks, and explore advanced concepts in robotics and automation. Additionally, it provides a solid theoretical framework for troubleshooting, error analysis, and further advancements in robotic arm technology.

3.2. Features of Robotic Arm

The features of an Arduino-based robotic arm include:

1. Programmability: It can be easily programmed and controlled using the Arduino platform, allowing for custom sequences and tasks.

2. Flexibility: The arm offers multiple degrees of freedom, enabling it to perform complex and precise movements.

3. Sensor Integration: It can be integrated with various sensors to perceive and interact with the environment, enhancing its functionality.

4. Open-Source: The Arduino platform is open-source, fostering collaboration, innovation, and a vast community for support and knowledge sharing.

5. Customizability: Users can modify and adapt the arm's design, control algorithms, and functionality to suit their specific requirements or experiment with different components and sensors.

3.3. Advantages of Robotic Arm

The advantages of an Arduino-based robotic arm are as follows:

1. Affordability: Arduino-based solutions are relatively low-cost, making robotic arm projects more accessible for hobbyists, students, and small-scale applications.

2. Ease of Use: The Arduino platform offers a user-friendly interface, simplifying programming, control, and integration of the robotic arm.

3. Community Support: The Arduino community provides extensive resources, tutorials, and libraries, enabling users to troubleshoot, learn, and collaborate.

4. Customizability: Arduino-based robotic arms can be easily modified and expanded, allowing for personalized designs, sensor integration, and functionality enhancements.

5. Educational Value: The combination of Arduino and robotic arm technology facilitates hands-on learning, fostering understanding of robotics, programming, and automation principles.

3.4. Disadvantages of Robotic Arm

While Arduino-based robotic arms offer numerous advantages, there are a few potential disadvantages to consider:

1. Processing Power Limitations: Arduino boards have limited processing power, which can restrict the complexity and speed of computations required for advanced robotic tasks.

2. Memory Constraints: Arduino boards often have limited memory capacity, which can limit the storage and execution of large-scale programs or datasets.

3. Hardware Limitations: Arduino platforms may have limited input/output pins or lack specialized hardware interfaces, which can restrict the connectivity and integration of advanced sensors or actuators.

4. Real-Time Constraints: Arduino boards may not always provide real-time control capabilities, making them less suitable for applications that require precise timing or rapid response.

5. Scalability: Arduino platforms may have limitations in terms of scalability, making them less suitable for large-scale industrial automation or heavy-duty applications that require high power or larger payloads.

It's important to consider these potential disadvantages and assess whether they align with the specific requirements and expectations of the intended application before choosing an Arduino-based robotic arm.

3.5. Application areas of Robotic Arm

Arduino-based robotic arms find applications in various fields, including:

1. Education: They serve as educational tools for teaching robotics and programming concepts.

2. Research: Robotic arms are used in research projects exploring automation, human-robot interaction, and artificial intelligence.

3. Industrial Automation: They automate tasks such as assembly, pick-and-place operations, or material handling in manufacturing industries.

4. Healthcare: Robotic arms can be employed in surgical procedures, rehabilitation, and assistive devices for individuals with disabilities.

5. Agriculture: They can be used for automated farming tasks, such as planting, harvesting, or sorting crops.

6. Home Automation: Robotic arms can be integrated into smart home systems for tasks like cleaning, object manipulation, or personal assistance.

7. Prototyping and Hobbyist Projects: Arduino-based robotic arms are popular among hobbyists and makers for prototyping, experimentation, and creative projects.

3.6. Robotic Arm Technologies

Robotic arm technologies have witnessed significant advancements, enabling them to perform complex tasks with precision and efficiency. Here are some notable technologies in the field of robotic arms:

1. Kinematic Redundancy: Robotic arms equipped with more degrees of freedom than necessary for a task exhibit kinematic redundancy. This allows for increased flexibility, adaptability, and avoidance of obstacles or joint limitations.

2. Force/Torque Sensing: Force/torque sensors integrated into robotic arms provide feedback on the forces and torques experienced during interactions with objects or

the environment. This enables tasks such as delicate object manipulation, compliant motion control, and force-based assembly.

3. Vision Systems: Robotic arms equipped with vision systems, including cameras and image processing algorithms, can perceive and analyze their surroundings. This facilitates object recognition, localization, and tracking, enabling tasks that require visual feedback or interaction with objects based on their appearance.

4. Collaborative Robotics (Cobots): Cobots are designed to work alongside humans safely. They incorporate sensors, such as proximity sensors or force sensors, and advanced control algorithms to detect and respond to human presence or contact, allowing for shared workspaces and cooperation.

5. Exoskeletons: Robotic exoskeletons are wearable devices that augment human strength and dexterity. These assistive robotic arms are used in rehabilitation, allowing individuals with motor impairments to regain or enhance their physical capabilities.

6. Machine Learning and AI: Integration of machine learning and artificial intelligence techniques enables robotic arms to learn from data and adapt their behavior. This leads to improved task performance, adaptability, and autonomous decision-making.

These advancements in robotic arm technologies continue to drive innovation in various industries, including manufacturing, healthcare, automation, and beyond, revolutionizing the way we interact with and benefit from robotic systems.

3.7. Project Layout

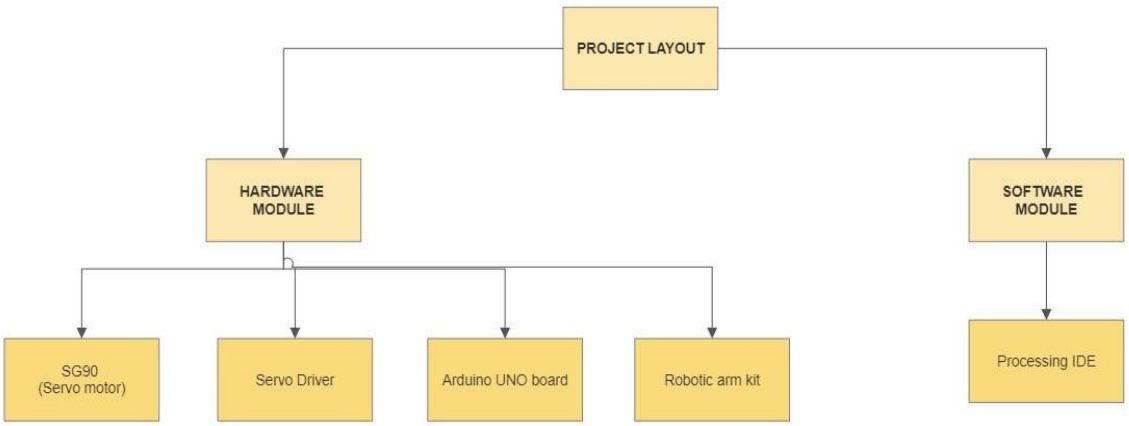


Figure 2. Layout of project module

Chapter 04: Hardware Requirements

4.1. Arduino Nano

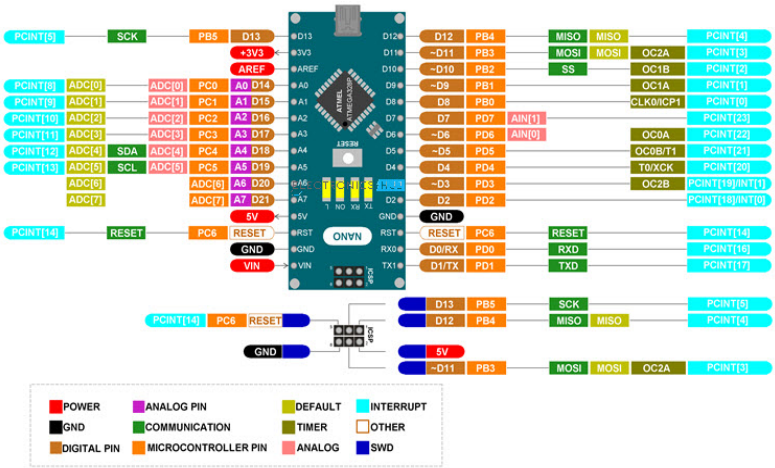
Arduino Nano is an open-source micro-controller board based on the microchip ATmega328p micro-controller. With the help of the controller, we can upload the program regarding the movements of the servos. By using the controller, we can control the number of servos at a time and synchronize the operation of the servo for -operation of any servo in any sequence and -to synchronize actuation of four servos sequentially in a loop programmed.

4.1.1. Features

The features of Arduino Nano ATmega328p includes the following:

- The operating voltage is 5V
- The recommended input voltage will range from 7v to 12V
- The input voltage ranges from 6v to 20V
- Digital input/output pins are 14
- Analog i/p pins are 6
- DC Current for each input/output pin is 40 m A
- DC Current for 3.3V Pin is 50 mA
- Flash Memory is 32 KB
- SRAM is 2 KB
- EEPROM is 1 KB
- CLK Speed is 16 MHz

4.1.2. Pin Configuration



Sl. No.	Pin name	Type	Function
1.	D1/TX	I/O	Serial TX Pin (Digital I/O Pin)
2.	D0/RX	I/O	Serial RX Pin (Digital I/O Pin)
3.	RESET	Input	Reset (Active low)
4.	GND	Power	Supply Ground
5.	D2-D13	I/O	Digital I/O Pin
6.	3V3	Output	+3.3V Output (from FTDI)
7.	AREF	Input	ADC reference
8.	A0-A7	Input	Analog Input Channel
9.	+5V	Output or Input	+5V (Output from on-board Regulator) or +5V (Input from External Power Supply)
10.	Vin	Power	Supply Voltage

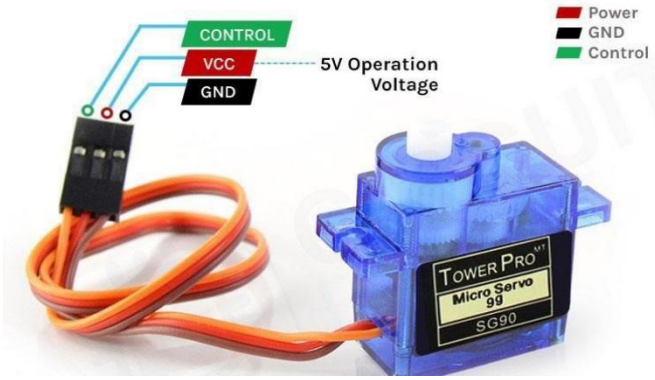
4.2. Servo Motor(SG90)

SG90 is a popular micro servo motor commonly used in hobbyist and DIY projects. It is a small, low-cost servo motor that can rotate 180 degrees with a maximum torque of 1.8 kg-cm. It operates at 4.8-6V and has a weight of approximately 9 grams, making it ideal for small-scale robotics and model control applications.

4.2.1. Features

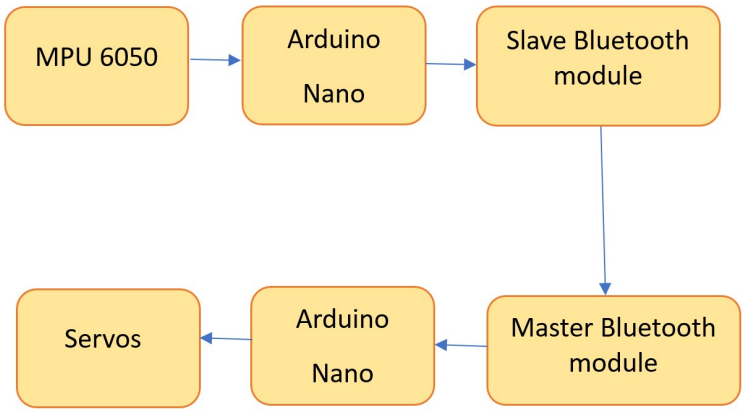
- Torque: 2.0kg/cm(4.8V), 2.2kg/cm(6V)
- Speed: 0.09s/60°(4.8V), 0.08s/60°(6V)
- Rotate angle: 180°
- Operating voltage: 4.8 ~ 6V
- Gear: plastic
- Dead band: 7us
- Weight: 10.5g
- Dimension: 22.8mm × 12.2mm × 28.5mm

4.2.2. Pin Configuration



SG90 Servo Motor Pinouts	
Terminal Color	**Brief Description**
Red	VCC Connected to 3.5V to 5V
Brown	Ground
Orange/Yellow	Control Signal or PWM Signal to be appiled on this terminal

4.3. Block diagram of the purposed system



4.3.1 Working of the system

The robotic manipulator works on the principle of electrical input energy to perform some mechanical work effectively with the help of some automation and program based.

Rotary joint – links move in rotary fashion concerning its joint when actuated. The robotic arm consists of a servo motor which is used for angular rotations. This servomotor used works on the principle of Fleming's right hand rule and is controlled using an Arduino circuit board. Degrees of freedom is defined as the ability of a joint to produce linear or rotary movement when actuated. The number of DOF for a robot is equal to the number of joint axes in the robotic arm.

After writing the code in the sketch then we uploaded into the Arduino NANO board using the cable then the micro-controller sends instructions to the PWM servo driver to control the servo motors simultaneously at the joints of the arm. Most of the control is based on "servo libraries", we can control servo motor. At last, the joints of the robotic arm move in 4 DOF according to the code.

4.3.2 Components Required

Sl. No.	Component and Specification	Quantity
1.	Arduino Nano	2
2.	Servo motor (SG90)	5
3.	Jumper wires	As required
4.	Push button	1
5.	5V DC female jack	1
6.	Sun Board	As required
7.	Potentiometers	4
8.	Bluetooth	2
9.	MPU6050	1
10.	Battery pack	1

Table 1. Component listing.

Chapter 05: Software Requirements

5.1. Arduino IDE (Embedded C / C++)

Arduino IDE is an open-source software, designed by Arduino.cc and mainly used for writing, compiling & uploading code to almost all Arduino Modules. It is an official Arduino software, making code compilation too easy that even a common person with no prior technical knowledge can get their feet wet with the learning process. It is available for all operating systems i.e., MAC, Windows, Linux and runs on the Java Platform that comes with inbuilt functions and commands that play a vital role in debugging, editing and compiling the code. A range of Arduino modules available including Arduino Uno, Arduino Mega, Arduino Leonardo, Arduino Micro and many more. Each of them contains a microcontroller on the board that is actually programmed and accepts the information in the form of code. The main code, also known as a sketch, created on the IDE platform will ultimately generate a Hex File which is then transferred and uploaded in the controller on the board. The IDE environment mainly contains two basic parts: Editor and Compiler where former is used for writing the required code and later is used for compiling and uploading the code into the given Arduino Module. This environment supports both C and C++ languages.

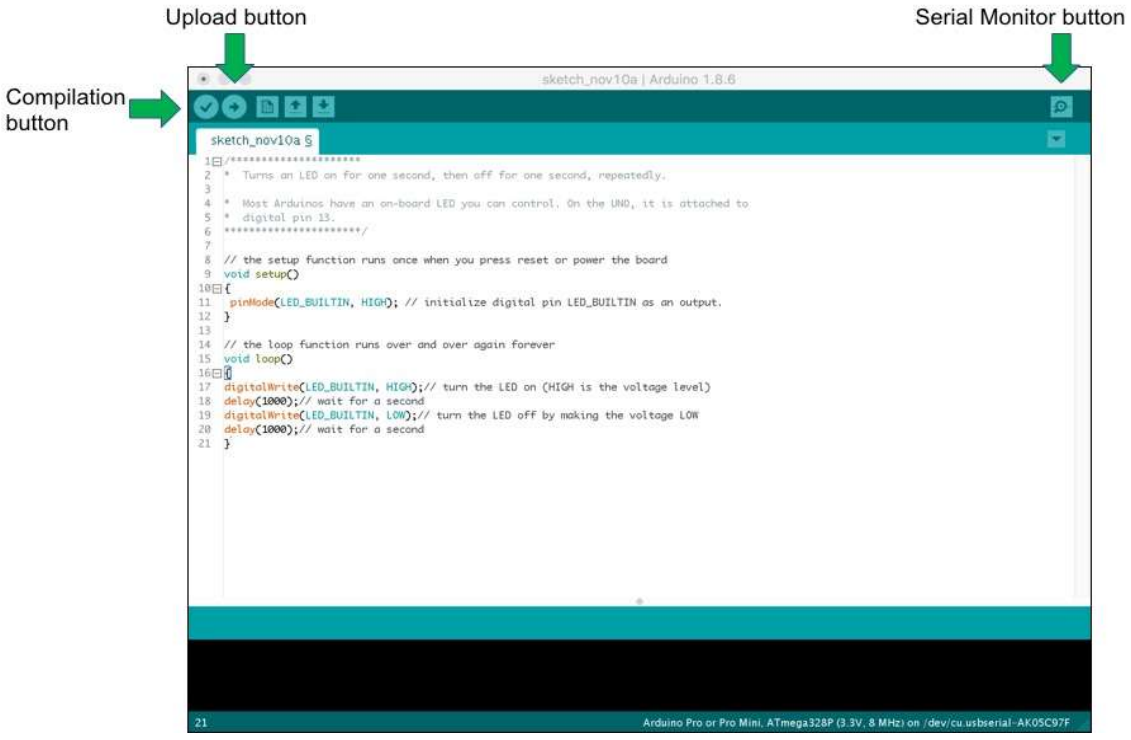
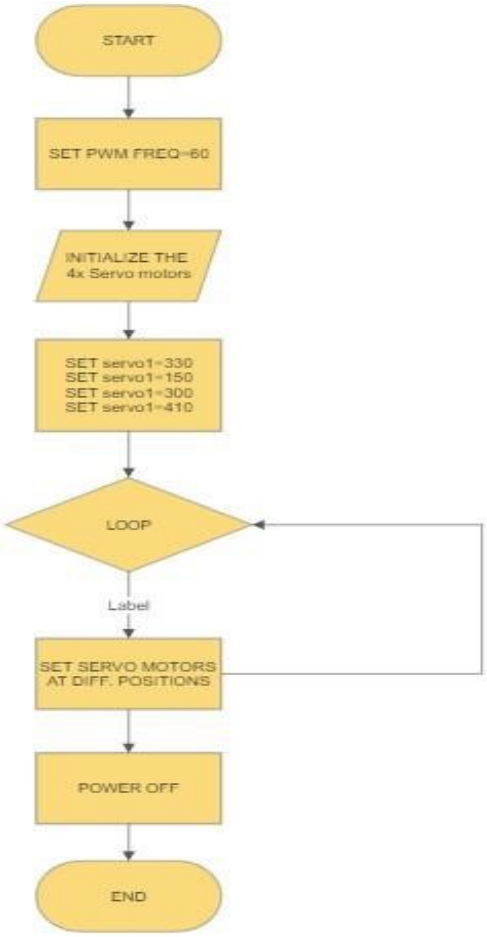
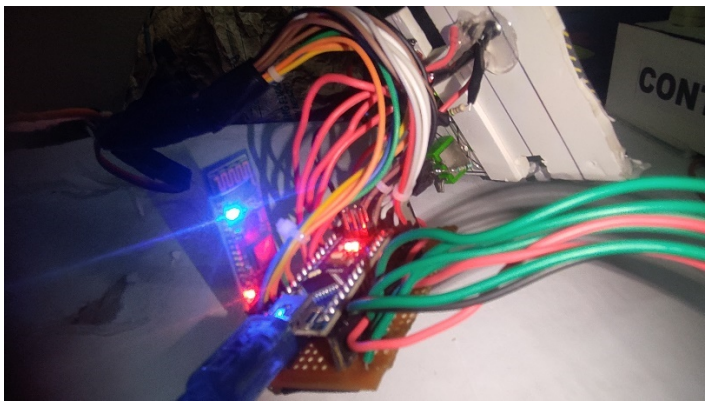


Figure 5.1. The Arduino IDE Interface

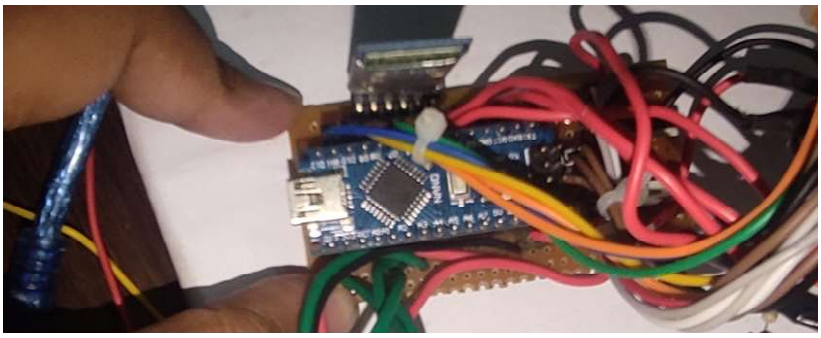
5.2. Logic and Flowchart



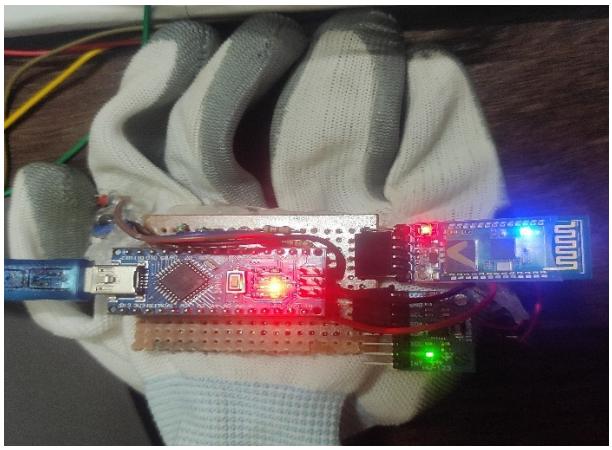
Chapter 06: Project development & Testing Aspects



(Fig I-circuitry)



(Fig II-Arm board)



(Fig III-Glove circuit)

Chapter 07: Conclusion & Future Scope

7.1. Result

The results of using robotic arms can be significant in terms of increased productivity, improved accuracy and precision, enhanced safety, greater flexibility, cost savings, and scalability. Robotic arms can work continuously without getting tired or making mistakes, which can lead to increased production output and efficiency. They can also improve safety in hazardous environments, and reduce the risk of workplace injuries. Additionally, robotic arms can be programmed to perform a wide range of tasks, which makes them flexible and adaptable to changing production needs. Overall, the use of robotic arms can result in increased profitability and competitiveness for businesses, as well as improved quality of products and services.

7.2. Conclusion

The Arduino-based robotic arm project successfully utilized servo motors and potentiometers to create a functional and precise robotic arm system. The integration of servo motors enabled accurate and controlled movements of the arm, while the potentiometers provided real-time feedback and positional data. Through software programming, the Arduino micro-controller translated the potentiometer data into servo motor commands, allowing the arm to replicate human-like movements. Overall, the project showcased the potential of combining hardware components and software programming to create a versatile and capable robotic arm system.

7.3. Limitations

Robotic arms have limitations including high cost, complex programming, space requirements, limited adaptability, maintenance and repair needs, safety concerns, limited dexterity, and dependence on power and technology. These limitations can impact their use in various industries, and careful consideration of costs and benefits is necessary for successful implementation.

7.4. Further Enhancement and Future Scope

Robotic Arms have a wide scope of development. Shortly the arms will be able to perform every task as humans and in a much better way. Brain Computer Interface can be used to acquire signals from the human brain and control the arm. The system can work in the same way as a human arm. We can add multiple sensors in order to sense the object and automatically pickup item. Rather than picking of items we can welding hand or any industrial machine can fit into it.

References

https://circuitdigest.com/microcontroller-projects/diy-hand-gesture-controlled-robotic-arm-using-arduino-nano

https://smartbuilds.io/diy-robot-arm-arduino-hand-gestures/

https://techatronic.com/hand-gesture-controlled-robot-arm-using-arduino/

Appendix 01

A01.1. Code Listing

1. Arm code.
2. Controller code.

A01.2. Main Code

1. Arm code.

```
#define b A0
#define s A1
#define e A2
#define w A3
#include<SoftwareSerial.h>
SoftwareSerial bt(7,8);//(rx,tx)
#include <Servo.h>
Servo C;
Servo W;
Servo E;
Servo S;
Servo B;
int prev_wr=0,prev_el=0,prev_so=0,prev_ba=0,c=0,OP=0;

int bw=0,be=0,bs=0,bb=0;
int v=0,v1=0,v2=0,v3=0,v4=0,v5=0;



void setup() {
  // put your setup code here, to run once:
  C.attach(11);
  W.attach(10);
```

```
 E.attach(9);
 S.attach(6);
 B.attach(5);

 pinMode(12,INPUT_PULLUP);
 pinMode(3,OUTPUT);

 Serial.begin(9600);
 bt.begin(9600);
 attachInterrupt(digitalPinToInterrupt(2),buttonPressed1,RISING);  //  function for creating external interrupts at pin2 on Rising (LOW to HIGH)

}

 void loop()
 {
 // put your main code here, to run repeatedly:
  if (OP)
  gesture();
  else
  controller();
 }
 void gesture()
 {

  if(bt.available())
   v=bt.parseInt();


    v1=v%10;
    v=v/10;
```

```
   v2=v%10;
   v=v/10;

 if(v1>=0&&v1<=1)
  v3=v1;
 if(v2>=1&&v2<=8)
  v4=v2;
 if(v>=0&&v<=300){
  v5=v;}

switch (v4){
  case 1:
  bs=v5;
  be=be;
  bw=bw;
  bb=bb;
  break;
  case 3:
  bs=bs;
  be=v5;
  bw=bw;
  bb=bb;
  break;
  case 5:
  bs=bs;
  be=be;
  bw=v5;
  bb=bb;
  break;
  case 7:
  bs=bs;
```

```
    be=be;
    bw=bw;
    bb=v5;
    break;
    default:
    bs=bs;
    be=be;
    bw=bw;
    bb=bb;
    break;

  }
   int Wt=map(bw,0,300,145,15);
   int Et=map(be,0,300,155,15);
   int St=map(bs,0,300,115,55);
   int Bt=map(bb,0,300,145,10);

   posi(Wt,Et,St,Bt,v3);
 }
void controller()
{
   int Wv=map(analogRead(w),0,1023,0,180);
   int Ev=map(analogRead(e),0,1023,0,180);
   int Sv=map(analogRead(s),0,1023,55,115);
   int Bv=map(analogRead(b),0,1023,0,180);

   if(digitalRead(12)==0){
   delay(50);
   if(digitalRead(12)==1)
   c=!c;
   }
```

```
   pos(Wv,Ev,Sv,Bv,c);

 }

void posi(int wr,int el,int so,int ba,int c)
 {

 Serial.print("Wrist:-");
 Serial.print(wr);
 Serial.print(" Elbo:-");
 Serial.print(el);
 Serial.print(" Soulder:-");
 Serial.print(so);
 Serial.print(" Base:-");
 Serial.print(ba);
 Serial.print(" claw:-");
 Serial.println(c);
 //delay(1000);

 if(wr>prev_wr){
  for(int i=prev_wr;i<=wr;i++){
 W.write(i);
 delay(10);
 }
  prev_wr=wr;
 }
 if(wr<prev_wr){
  for(int i=prev_wr;i>=wr;i--){
    W.write(i);
    delay(10);
```

```
}
 prev_wr=wr;
}
  if(el>=prev_el){
    for(int i=prev_el;i<=el;i++){
     E.write(i);
     delay(10);
     }
     prev_el=el;
     }
  if(el<=prev_el){
    for(int i=prev_el;i>=el;i--){
     E.write(i);
     delay(10);
     }
     prev_el=el;
     }

 if(so>=prev_so){
  for(int i=prev_so;i<=so;i++){
    S.write(i);
    delay(10);
  }
  prev_so=so;
 }
  if(so<=prev_so){
  for(int i=prev_so;i>=so;i--){
    S.write(i);
    delay(10);
  }
 prev_so=so;
```

```
 }
 if(ba>=prev_ba){
  for(int i=prev_ba;i<=ba;i++){
    B.write(i);
  delay(10);
  }
 prev_ba=ba;
 }
 if(ba<=prev_ba){
  for(int i=prev_ba;i>=ba;i--){
    B.write(i);
  delay(10);
  }
 prev_ba=ba;
 }

 if(c){
  C.write(165);
 }
 else
 C.write(40);

}

 void buttonPressed1()          //ISR function excutes when push button at pinD2 is pressed
 {
 OP=!OP;
 digitalWrite(3,OP);}
```

2. Controller code.

```
 #include<Wire.h>               //I2C Wire Library
#include<SoftwareSerial.h>
#define claw 2
#define select 3
#define shoulder 5
#define elbo 7
#define wrist 9

SoftwareSerial bt(11,10);//(rx,tx)
const int MPU_addr=0x68;        //MPU6050 I2C Address

int16_t axis_X,axis_Y,axis_Z;

int minVal=265;

int maxVal=402;

int c=0;

int pos=0;

double x;

double y;

double z;

int trans1;
long int v;
```

```
void setup()

{

  Serial.begin(9600);

   bt.begin(9600);

  Wire.begin();                  //Initilize I2C Communication

  Wire.beginTransmission(MPU_addr);  //Start communication with MPU6050

  Wire.write(0x6B);                //Writes to Register 6B

  Wire.write(0);                 //Writes 0 into 6B Register to Reset

  Wire.endTransmission(true);       //Ends I2C transmission
  pinMode(claw,INPUT_PULLUP);
  pinMode(select,INPUT_PULLUP);
  pinMode(5,OUTPUT);
  pinMode(7,OUTPUT);
  pinMode(9,OUTPUT);

  attachInterrupt(digitalPinToInterrupt(2),buttonPressed1,RISING);  //  function for creating external interrupts at pin2 on Rising (LOW to HIGH)

  attachInterrupt(digitalPinToInterrupt(3),buttonPressed2,RISING);  //  function for creating external interrupts at pin3 on Rising (LOW to HIGH)
```

```
}

void loop()

{

  Wire.beginTransmission(MPU_addr);

  Wire.write(0x3B);                //Start with regsiter 0x3B

  Wire.endTransmission(false);

  Wire.requestFrom(MPU_addr,14,true);  //Read 14 Registers



  axis_X=Wire.read()<<8|Wire.read();              //Reads the MPU6050 X,Y,Z AXIS Value

  axis_Y=Wire.read()<<8|Wire.read();

  axis_Z=Wire.read()<<8|Wire.read();



   int xAng = map(axis_X,minVal,maxVal,-90,90);       // Maps axis values in terms of -90 to
+90

  int yAng = map(axis_Y,minVal,maxVal,-90,90);

  int zAng = map(axis_Z,minVal,maxVal,-90,90);
```

```
 x= RAD_TO_DEG * (atan2(-yAng, -zAng)+PI);       //Formula to convert into degree

 y= RAD_TO_DEG * (atan2(-xAng, -zAng)+PI);

 z= RAD_TO_DEG * (atan2(-yAng, -xAng)+PI);

if(c)
digitalWrite(13,HIGH);
else
digitalWrite(13,LOW);

       switch(pos){
         case 1:
         digitalWrite(5,HIGH);
         digitalWrite(7,LOW);
         digitalWrite(9,LOW);
          trans1 = x_move() ;

         break;
         case 3:
          digitalWrite(5,LOW);
          digitalWrite(7,HIGH);
          digitalWrite(9,LOW);
          trans1 = x_move();

          break;
         case 5:
          digitalWrite(5,LOW);
          digitalWrite(7,LOW);
```

```
          digitalWrite(9,HIGH);
          trans1 = x_move();

          break;
        case 7:
          digitalWrite(5,HIGH);
          digitalWrite(7,HIGH);
          digitalWrite(9,HIGH);
          trans1 = y_move();

          break;
          default:
              digitalWrite(5,LOW);
              digitalWrite(7,LOW);
              digitalWrite(9,LOW);
       }

      v=(trans1*100)+(pos*10)+c;
      bt.println(v);
      delay(700);


      Serial.println(v);
     delay(100);

}
long int x_move(){
  long int mov1,val;
 if(x >=0 && x <= 50)
 {
   mov1 = map(x,0,50,150,300);
```

```
  }
  else if(x >=280 && x <= 360)

  {
     mov1 = map(x,360,280,150,0);

  }
  else if(x>50&&x<200)
    mov1=300;
 else if(x<280&&x>100)
    mov1=0;

return mov1;

}
long int y_move(){
  long int mov3 ;

  if(y >=0 && y <= 40)

  {
     mov3 = map(y,0,40,99,0);
  }

  else if(y >=280 && y <= 360)

  {
      mov3 = map(y,360,280,100,300);
  }
  else if(y <=280 && y >= 180)
```

```
   mov3=300;
  else if(y >=40 && y <= 200)
   mov3=0;
  return mov3;
}

void buttonPressed1()          //ISR function excutes when push button at pinD2 is pressed

{
c=!c;

}

void buttonPressed2()          //ISR function excutes when push button at pinD3 is pressed

{
  pos++;
  if(pos>7)
  pos=0;

}
```

A01.3. Libraries

```
#include<SoftwareSerial.h>
#include <Servo.h>
#include<Wire.h>
```

Appendix 02

A02.1. Public Repository Note

This report documents an academic team project. Personal and institutional identifiers have been removed from this public copy. The project was developed collaboratively; this repository does not claim sole authorship.

A02.2. Project Management

#	Component	Contributor A	Contributor B	Repository Contributor	Total
1.	Planning	50	25	25	100
2.	Background research and analysis	40	30	30	100
3.	Hardware design	40	30	30	100
4.	Software design	35	35	30	100
5.	Testing	35	30	35	100
6.	Final assembling	40	30	30	100
7.	Project report writing	30	40	30	100
8.	Presentation	35	35	30	100
9.	Logistics	40	30	30	100

A02.3. Bill of Material

Table 1. Component listing.

#	Component	Specification	Unit Cost	Quantity	Total
1.	Arduino Board	Nano	350	2	700
2.	Sunboard	2ft x 1ft	60	1	60
3.	Potentiometer	10kΩ	12	4	48
4.	Servo motor	SG90	120	5	600
5.	Push button	----------	4	1	4
6.	5V DC female jack	----------	10	1	10
7.	12 V Adapter	12v 1amp	120	1	120
8.	wires	23swg	8	3mts	24

9.	Jumper	Male to male	30	1 set	30
10.	Switch	SPDT	10	3	30
11.	Resistors	----------	1	10	10
12.	LED	----------	2	7	14
13.	MPU6050	----------	150	1	150
14.	Battery with Holder	----------	55,20	2,1	130
15.	Bluetooth module	HC-05	250	2	500
6.	Gloves	----------	50	1	50
				Grand Total	2480

Appendix 03

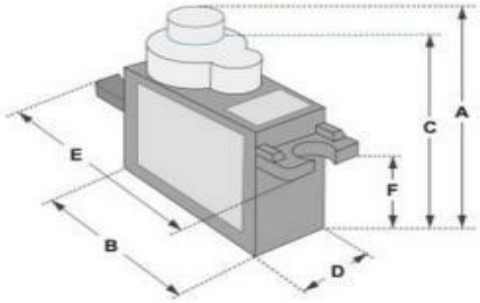
A03.1. Data Sheets

Servo Motor:

SERVO MOTOR SG90 DATA SHEET

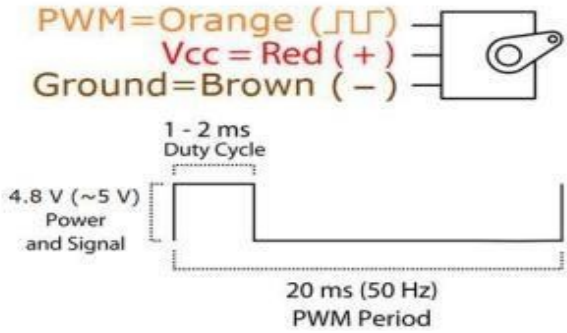


Tiny and lightweight with high output power. Servo can rotate approximately 180 degrees (90 in each direction), and works just like the standard kinds but smaller. You can use any servo code, hardware or library to control these servos. Good for beginners who want to make stuff move without building a motor controller with feedback & gear box, especially since it will fit in small places. It comes with a 3 horns (arms) and hardware.



Dimensions & Specifications
A (mm) : 32
B (mm) : 23
C (mm) : 28.5
D (mm) : 12
E (mm) : 32
F (mm) : 19.5
Speed (sec) : 0.1
Torque (kg-cm) : 2.5
Weight (g) : 14.7
Voltage : 4.8 - 6

Position "0" (1.5 ms pulse) is middle, "90" (~2ms pulse) is middle, is all the way to the right, "-90" (~1ms pulse) is all the way to the left.



Arduino Nano:

Arduino® Nano

Product Reference Manual
SKU: A000005



Description

Arduino® Nano is an intelligent development board designed for building faster prototypes with the smallest dimension. Arduino Nano being the oldest member of the Nano family, provides enough interfaces for your breadboard-friendly applications. At the heart of the board is **ATmega328 microcontroller** clocked at a frequency of 16 MHz featuring more or less the same functionalities as the Arduino Duemilanove. The board offers 22 digital input/output pins, 8 analog pins, and a mini-USB port.

Target Areas

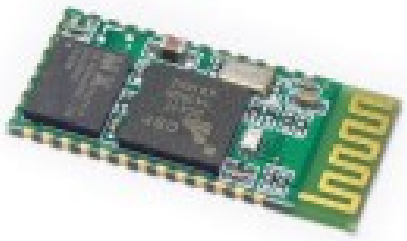
Maker, Security, Environmental, Robotics and Control Systems

HC-05 :

HC-05

-Bluetooth to Serial Port Module

Overview



HC-05 module is an easy to use Bluetooth SPP (Serial Port Protocol) module, designed for transparent wireless serial connection setup.
Serial port Bluetooth module is fully qualified Bluetooth V2.0+EDR (Enhanced Data Rate) 3Mbps Modulation with complete 2.4GHz radio transceiver and baseband. It uses CSR Bluecore 04-External single chip Bluetooth system with CMOS technology and with AFH(Adaptive Frequency Hopping Feature). It has the footprint as small as 12.7mmx27mm. Hope it will simplify your overall design/development cycle.

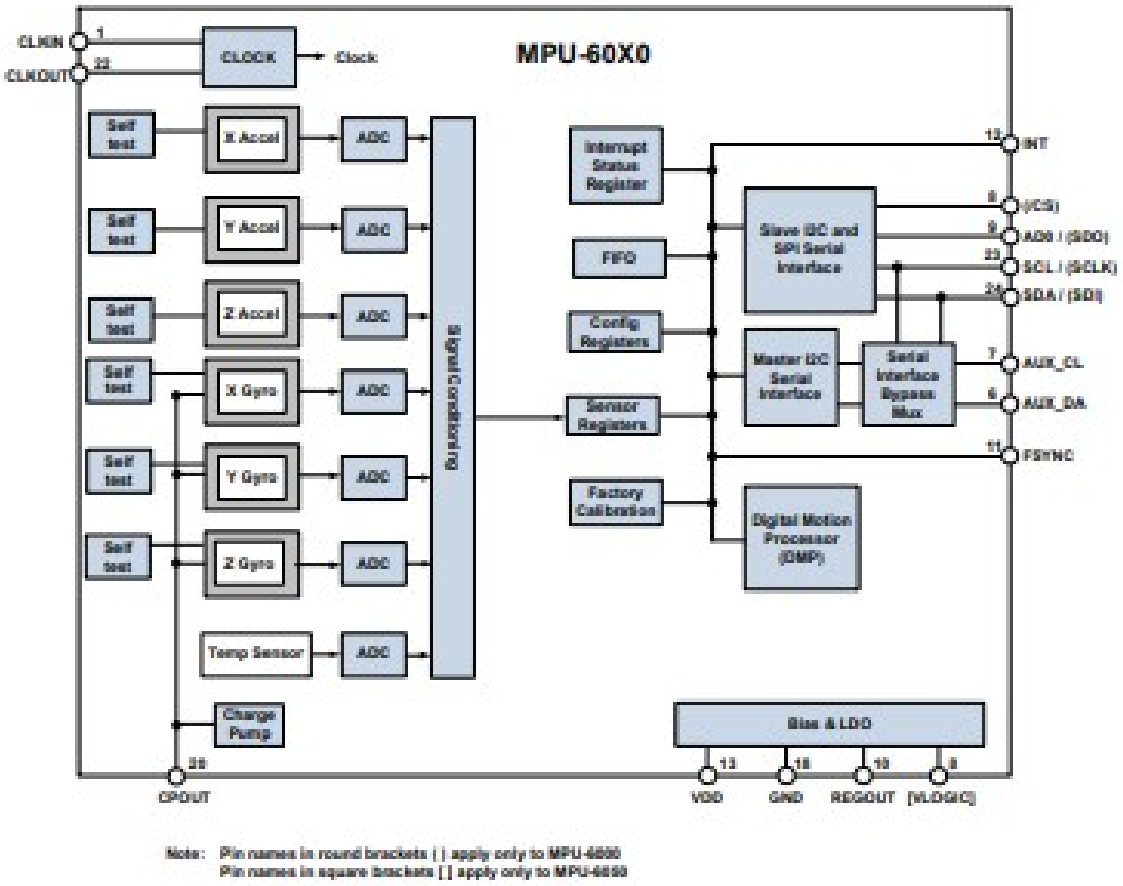
Specifications

Hardware features

- Typical -80dBm sensitivity
- Up to +4dBm RF transmit power
- Low Power 1.8V Operation ,1.8 to 3.6V I/O
- PIO control
- UART interface with programmable baud rate
- With integrated antenna
- With edge connector

MPU-6050 :

7.5 Block Diagram



Note: Pin names in round brackets ( ) apply only to MPU-6000
Pin names in square brackets [ ] apply only to MPU-6050

7.6 Overview

The MPU-60X0 is comprised of the following key blocks and functions:

- Three-axis MEMS rate gyroscope sensor with 16-bit ADCs and signal conditioning
- Three-axis MEMS accelerometer sensor with 16-bit ADCs and signal conditioning
- Digital Motion Processor (DMP) engine
- Primary I^2C and SPI (MPU-6000 only) serial communications interfaces
- Auxiliary I^2C serial interface for 3rd party magnetometer & other sensors
- Clocking
- Sensor Data Registers
- FIFO
- Interrupts
- Digital-Output Temperature Sensor
- Gyroscope & Accelerometer Self-test
- Bias and LDO
- Charge Pump